Team Project - Concise Guide

Roles, process, deliverables, and grading

ASE

ASE @ Northern Kentucky University

1. Goal and Scope

Work as a professional software team to refactor and extend the provided Python/PyGame Tetris prototype. The team must deliver working, tested, maintainable software and clear evidence of a professional process.

- **Sprint 1:** use the required Venv/PIP workflow and produce a working MVP.
- **Sprint 2:** use the required UV workflow and improve the product through design patterns, SOLID principles, and refactoring.

2. Roles

2.1. Team Leader

- Coordinate the schedule, task ownership, integration, and risks.
- Manage integration, regression, and acceptance testing.
- Compile documentation and present the weekly team status.
- Submit shared team deliverables and evaluate each member using evidence.
- Delegate the weekly presentation if absent.

2.2. Team Members

- Own agreed features and testable requirements.
- Write code, unit tests, and contribution documentation.
- Update the individual Canvas contribution page at least 24 hours before the first weekly meeting.
- Review code and help with integration, regression, and acceptance tests.
- Report blockers early and submit private self, leader, and peer evaluations.

A team has 3-4 members (5 only in a truly special case). The whole team agrees on written rules, review practices, responsibilities, and consequences before development begins.

3. Process and Timeline

- **Weeks 1-3:** team formation, tools, repository, Canvas pages, problem, requirements, architecture, tests, milestones, weekly schedule, and team rules
- **Week 4:** Project Plan Presentation (PPP) and HW2 planning deliverables
- **Sprint 1 (from Week 5 until the first midterm):** build and test the prototype/MVP
- **Before S1P:** S1R and HW4 preparation/submission, shortly before S1P
- **Right after the first midterm (about Week 9 or 10):** Sprint 1 Presentation (S1P)
- **Sprint 2 (after S1P, until Week 16):** final implementation, refactoring, design improvement, testing, and documentation
- **Before FP:** S2R shortly before FP, final checks, and evidence-based evaluations
- **Week 16:** final submission and Final Presentation (FP)

These are the practice story's process weeks, not confirmed calendar dates. Canvas is the official source for exact dates, deadlines, and changes around breaks. Do not automatically schedule a classroom presentation during Spring Break.

3.1. Ceremonies

1. PPP - Project Plan Presentation
2. S1R - Sprint 1 Retrospective
3. S1P - Sprint 1 Presentation
4. S2R - Sprint 2 Retrospective
5. FP - Final Presentation

4. Five Ceremonies: What to Prepare and Produce

PPP means Project Plan Presentation, not Project Preparation Project. S1P is a presentation, not sprint planning. There is no separate S2P in this five-ceremony model: FP reviews the final product and both sprints.

4.1. PPP - Project Plan Presentation (W4)

Audience: classmates, instructor, and other stakeholders; every team member presents their part. **Purpose:** show that the problem matters and that the team has a feasible plan.

- Explain the problem and users; connect problems to features and testable user stories (who, what, why).
- Show acceptance criteria, data model, architecture, feature ownership, both sprint goals, milestones, weekly plan, and agreed team rules.
- Prepare a project plan and Marp slides/PDF; store artifacts in GitHub and link them from Canvas. Complete HW2 according to its assignment instructions.
- **Outcome:** a shared plan and clear commitments; incorporate stakeholder feedback before Sprint 1 development.

4.2. S1R - Sprint 1 Retrospective (before S1P)

Audience: the development team only; led by the team leader. **Purpose:** improve the process, not demonstrate the product or blame people.

- Discuss what went well, what did not, and what should change.
- Record specific improvement actions with owners and follow-up targets; update the Sprint 2 plan and backlog.
- Consider dogfooding: use your own software to discover usability problems, missing features, and bugs.
- **Timing:** shortly before S1P; Canvas controls the actual schedule.

4.3. S1P - Sprint 1 Presentation (right after the first midterm)

Audience: stakeholders; every member presents their contribution. **Purpose:** review the working prototype/MVP and agree on the next commitments.

- Before presenting, finish S1R and the HW4 materials: current code/tests, user manual, architecture/design/API documentation where relevant, demo, slides/PDF, and revised Sprint 2 plan.
- Compare promised and completed features/requirements; show metrics, working software, challenges, solutions, and lessons learned.
- Present Sprint 2 priorities and the changes resulting from S1R and stakeholder feedback. Explain previously reported delays honestly: no surprises.
- The time depends on the situation and need not be long (for reference, about 5 minutes plus 5 minutes of Q&A). Prepare a recorded backup for a live demo; Canvas controls the actual time allocation.

4.4. S2R - Sprint 2 Retrospective (before FP)

Audience: the development team only. Reflect on Sprint 2 and the whole project: communication, planning, reviews, testing, quality, and dogfooding results.

Record final corrective actions and lessons for future projects. Prepare the final rehearsal and checks. S2R is held shortly before FP, subject to Canvas scheduling.

4.5. FP - Final Presentation (W16)

Audience: stakeholders; all team members present. Show the complete semester's result, not just Sprint 2.

- Compare the original/revised commitments with delivered features and acceptance-test evidence.
- Demonstrate the product, improvements since S1P, design quality, challenges, and learning.
- Publish slides/PDF and supporting artifacts before class and link them from the Canvas team page.
- Complete final submissions and evaluations by their separate Canvas deadlines; presentation and submission need not be on the same day.

5. Shared Concepts

- **Prototype:** tests feasibility and reduces unknowns quickly. **MVP:** Minimum Viable Product, a small working product with the necessary design, tests, and documentation.
- **Feature:** user-visible capability. **Requirement:** a testable user story. **Acceptance criterion:** observable evidence that a requirement is satisfied.
- **Definition of Done:** implemented, reviewed, tested, documented, acceptance criteria met, and no critical known defects; agree on any deployment requirements as a team.
- **Deadline:** final due date. **Milestone:** intermediate checkpoint. **Sprint:** a bounded iteration with a goal and selected work, generally 1-4 weeks. This course uses two larger development iterations (Sprint 1 and Sprint 2).
- **Review/presentation:** inspect product outcomes with stakeholders. **Retrospective:** improve the team's process internally.
- **Quality:** maintainability and extensibility as well as useful features. Unit, integration, regression, and acceptance tests provide different evidence; counts and coverage alone are not proof of quality.

6. Weekly Progress

Each member reports:

- current milestone and whether it is on track
- work completed and work planned next
- completed/total features and requirements
- code and test metrics with GitHub links
- blockers, decisions needed, and a recovery plan when off track
- design principles, patterns, or refactorings applied

The leader converts individual updates into a concise team-level report for stakeholders. Do not publicly blame individuals. Handle repeated contribution problems separately with objective evidence and the professor's help. Report risks early: no surprises.

7. Required Deliverables

- One public shared GitHub repository with source code, tests, and documentation
- Team Canvas page and one individual contribution page per member
- Project plan, features, requirements, acceptance tests, architecture, milestones, weekly schedule, and team rules
- PPP, S1P, and FP slides in Marp Markdown and PDF
- User manual and architecture/design document in Marp Markdown and PDF
- Weekly progress reports with GitHub evidence
- GitHub.io project page and Hugo source
- Final team repository ZIP without generated files, unrelated files, or hidden `.git` data
- `tp_leader_deliverables_checklist.md`
- Required leader and member evaluation rubrics

8. Grading - 150 Points

Category	Points
Delivery, progress, and teamwork	100
Evaluations	50
Total	**150**

The point allocation is a guideline. The actual grading may differ by course, so always follow the current Canvas rubric. The evaluation forms use 100-point input rubrics. Their results are combined and normalized into the 50-point evaluation category. The instructor may adjust scores using GitHub artifacts, Canvas reports, presentations, and observed contributions.

Who evaluates whom: members evaluate themselves, peers, and the leader; the leader evaluates themselves and each member. Use observable contributions and artifacts, not friendship or vague impressions. The practice story's 40/30/30 product/process/team-dynamics model is likewise only a guideline; the current Canvas instructions decide.

9. Rules and Final Check

- Only the team leader submits shared deliverables unless Canvas says otherwise.
- Every member submits the required private evaluation rubric.
- Use pull requests and review before merging whenever practical.
- AI-assisted work must be understood and verified by the team and follow the course AI policy.
- Follow the deadline and extension policy on Canvas. Send an extension request at least 48 hours before the deadline.
- Before submission, verify all links, run the product and tests, open every PDF, review the ZIP contents, and confirm every required rubric is submitted.

Source: `story/src/team_project_practice.typ` and its `p_chapters` includes. The overview roadmap supplies the process weeks; conflicting short example sprint schedules in dialogue are not the semester schedule. The Tetris/tooling requirements and point totals above are course-specific guidelines, not universal rules from the story.